COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

07

PACKAGE 8 · INSTRUCTOR / SELF-STUDY

Answer key

Original core guidance · Worked transfer answers · Marking criteria

Undergraduate computer-engineering core

Modules 31-35 · Gate G

Edition: September 28, 2026

A guided self-study curriculum. Supplied examples are learning references, not completed learner projects. Simulation is the default; physical hardware is optional.

Contents

Page references and PDF bookmarks are linked for navigation.

Use after the attempt 3
Lesson 31.1 - Voltage, current, resistance and safe measurement 4
Lesson 31.2 - Capacitors, inductors, transients and frequency response 5
Lesson 31.3 - Diodes, transistors, CMOS and analog interfaces 6
Lesson 31.4 - Sampling, ADCs, DACs and noisy controller inputs 7
Lesson 32.1 - Boolean logic and combinational building blocks 8
Lesson 32.2 - Registers, finite-state machines and synchronous timing 9
Lesson 32.3 - Binary arithmetic, fixed point and overflow policies 10
Lesson 32.4 - Clock domains, reset and metastability 11
Lesson 33.1 - RTL semantics and synthesizable SystemVerilog 12
Lesson 33.2 - Testbenches, scoreboards and constrained stimulus 13
Lesson 33.3 - Assertions, coverage and formal reasoning 14
Lesson 33.4 - Synthesis, constraints and FPGA resource mapping 15
Lesson 34.1 - Instruction sets, datapaths and single-cycle execution 16
Lesson 34.2 - Pipelining, forwarding, stalls and branch recovery 17
Lesson 34.3 - Memory interfaces, caches and protection boundaries 18
Lesson 34.4 - Processor verification, toolchains and architectural evidence 19
Lesson 35.1 - Memory-mapped I/O, timers, interrupts and DMA 20
Lesson 35.2 - Framebuffers, scanout and pixel pipelines 21
Lesson 35.3 - A small accelerator and its software contract 22
Lesson 35.4 - Integrated digital-design review and optional board bring-up 23

Use after the attempt

The original core answers are preserved. New transfer answers include the principal reasoning and two-point marking guidance. Equivalent correct reasoning receives credit. Do not insist on identical wording.

A core answer earns 2 points when correct and sufficiently justified, 1 when substantially correct but incomplete, and 0 when wrong or missing. For numerical questions, units and the stated model matter. A correct number with a contradictory explanation should be revised. Core target: 5/6 for each lesson, together with completed study and a passing practical assignment. Transfer target: 3/4 as an additional diagnostic.

The key is not evidence that the learner implemented an RTL design, ran synthesis or completed the capstone. Actual lab and project evidence is assessed separately.

Lesson 31.1 - Voltage, current, resistance and safe measurement

Core quiz - original program questions / 6 points

31.1-C1. What current flows through 1,000 ohms at 5 V?

Answer: 0.005 A, or 5 mA.

31.1-C2. What power is dissipated in that ideal resistor?

Answer: 5 V x 0.005 A = 0.025 W, or 25 mW.

31.1-C3. Why can putting a meter in the wrong current-measurement configuration be dangerous?

Answer: It can create an unintended low-resistance path or short, damaging equipment or causing injury.

Transfer check - new application questions / 4 points

31.1-T1. A 5 V source drives R1=2 kohm above R2=3 kohm. A 6 kohm load is placed across R2. Find Vout and all three branch currents.

Answer and marking: R2 parallel RL is 2 kohm, so Vout=2.5 V. Top current is 1.25 mA; currents through R2 and RL are 0.833333 and 0.416667 mA. Their sum equals the top current. Award one point for loading the divider correctly and one for consistent currents and units.

31.1-T2. Why does testing 1,000 random resistor values inside a tolerance band not prove the worst-case divider voltage?

Answer and marking: A finite sample can miss the extreme combinations, and tolerance alone does not define a probability distribution. Analyze monotonicity and evaluate the corners; random sampling can supplement but not replace that bound. Award one point for the sampling limitation and one for the corner-analysis remedy.

Practical evidence that is still required

For the ideal Python model, absolute node-voltage error is at most 1e-9 V and current-balance residual at most 1e-12 A. Use a documented, looser tolerance for SPICE rather than claiming exact floating-point equality. Both tolerance extremes and the changed-load case must pass.

Lesson 31.2 - Capacitors, inductors, transients and frequency response

Core quiz - original program questions / 6 points

31.2-C1. What is tau for 10 kohm and 1 microfarad?

Answer: 10 ms.

31.2-C2. What fraction of a zero-to-final RC charging step is reached at one tau?

Answer: Approximately 1-exp(-1), or 63.2%.

31.2-C3. Why do initial conditions matter?

Answer: Energy already stored in capacitors or inductors affects the transient response.

Transfer check - new application questions / 4 points

31.2-T1. For the 10 kohm, 1 microfarad low-pass, find the 90% rise time and cutoff frequency.

Answer and marking: tau=0.01 s; t90=-tau ln(0.1)=0.0230259 s, and fc=1/(2 pi tau)=15.9155 Hz. One point for each result with units.

31.2-T2. Why is a forward-Euler RC response that overshoots at dt=1.5 tau not evidence that the ideal physical RC circuit overshoots?

Answer and marking: The Euler update has multiplier 1-dt/tau=-0.5, which creates alternating numerical error. The exact first-order passive RC response to this step is monotonic. Refine the step or use the exact interval update. One point for identifying numerical origin and one for the explanation/remedy.

Practical evidence that is still required

The ideal values agree within 0.1%; t90 is about 23.026 ms and fc about 15.915 Hz for the baseline. The Euler error must decrease as the step is refined over the tested stable range. Initial conditions, numerical step and units are explicit.

Lesson 31.3 - Diodes, transistors, CMOS and analog interfaces

Core quiz - original program questions / 6 points

31.3-C1. Why is a transistor not only a binary software switch?

Answer: Its electrical behavior is continuous and depends on voltages, currents, device properties and operating region.

31.3-C2. Does a logical high guarantee compatibility across voltage domains?

Answer: No. Electrical input limits and output levels must also match.

31.3-C3. Why distinguish absolute maximum ratings from recommended operating conditions?

Answer: Absolute limits are not a normal design target or a guarantee of intended performance.

Transfer check - new application questions / 4 points

31.3-T1. A non-inverting ideal gain is 10 on a nominal 0-to-3.3 V supply. Does a 0.5 V input establish a valid 5 V output?

Answer and marking: No. The linear equation requests 5 V, outside the supplied output range; a real device may saturate earlier and has other limits. One point for detecting the limit and one for distinguishing a requested ideal value from a physical guarantee.

31.3-T2. Starting low, a comparator sets at >=1.8 V and clears at <=1.5 V. What outputs follow 1.4, 1.6, 1.81, 1.7, 1.49 V?

Answer and marking: 0,0,1,1,0. Values in the middle band retain the previous state. One point for the sequence and one for explaining state retention.

Practical evidence that is still required

The simulated CMOS curve inverts over the educational range; the report identifies at least one limitation of the device model. The comparator sequence is exactly 0001110 under the defined equality convention, and an out-of-range amplifier case is rejected rather than reported as a valid output.

Lesson 31.4 - Sampling, ADCs, DACs and noisy controller inputs

Core quiz - original program questions / 6 points

31.4-C1. How many distinct codes does a 12-bit converter provide?

Answer: 4,096.

31.4-C2. Does 12-bit resolution guarantee 12-bit accuracy?

Answer: No. Noise and other errors can reduce accuracy and effective resolution.

31.4-C3. Why filter before sampling to prevent aliasing?

Answer: Once high-frequency content aliases into the sampled band, ordinary later filtering cannot distinguish it from genuine in-band content.

Transfer check - new application questions / 4 points

31.4-T1. For alpha=0.25, how many discrete updates are needed to reach at least 90% of a unit step from zero?

Answer and marking: Nine updates. Solve (0.75)^n<=0.1; n>=8.0039, so take the ceiling. Update 8 is about 0.899887, just below the target. One point for the result and one for the inequality/ indexing.

31.4-T2. At fs=1 kHz, why can digital processing alone not identify whether an observed 300 Hz sampled cosine came from 300 or 700 Hz?

Answer and marking: Those continuous cosines have identical samples at n/fs for this phase convention. The information was not preserved during sampling; an analog bandwidth constraint or other prior information is needed. One point for the identical samples and one for the information/ antialiasing explanation.

Practical evidence that is still required

All 4096 codes are representable under the stated convention; clipping and exact boundary cases are tested. The 90% threshold is first crossed on update 9 for the ideal alpha=0.25 step. Noise reduction is reported beside added delay, and calibration is not trained on the evaluation motion.

Lesson 32.1 - Boolean logic and combinational building blocks

Core quiz - original program questions / 6 points

32.1-C1. What distinguishes combinational from sequential logic?

Answer: Combinational outputs depend on current inputs without stored state; sequential behavior also depends on state.

32.1-C2. What does a two-to-one multiplexer do?

Answer: Selects one of two inputs according to a control bit.

32.1-C3. Does the shortest Boolean expression always imply the fastest circuit?

Answer: No. Physical mapping, fanout and timing also affect the implementation.

Transfer check - new application questions / 4 points

32.1-T1. A four-bit adder receives unsigned 13 and 7. Give the low result, carry-out and number of exhaustive input cases including carry-in.

Answer and marking: The full result is 20=10100 binary; low result is 4 and carry-out is 1. There are 16*16*2=512 cases. One point for the result/carry and one for the count.

32.1-T2. Why does a one-hot assertion not fully verify a 2-to-4 decoder?

Answer and marking: It checks how many outputs are selected, not which output corresponds to the input code. A design that always selects bit zero can pass one-hotness. Add input-to-output correspondence checks. One point for the distinction and one for the counterexample/check.

Practical evidence that is still required

All 512 adder input cases pass, all mux and decoder cases pass, and the independent oracle does not reuse the implementation recurrence. Invalid parameters are rejected and all output widths are explicit.

Lesson 32.2 - Registers, finite-state machines and synchronous timing

Core quiz - original program questions / 6 points

32.2-C1. What does a register capture in a synchronous model?

Answer: The value presented under the specified timing conditions at its active clock event.

32.2-C2. What simplified maximum frequency follows from an 8 ns period?

Answer: 125 MHz before additional uncertainty or margin.

32.2-C3. Does slowing the clock automatically fix hold violations?

Answer: No. Hold concerns the minimum-delay relationship near the capture edge.

Transfer check - new application questions / 4 points

32.2-T1. Given maximum delays 0.8+5.6+0.6 ns plus 0.5 ns uncertainty, what period is required and what is setup slack at 8 ns?

Answer and marking: The required period is 7.5 ns; slack at 8 ns is 0.5 ns. The corresponding idealized frequency limit is about 133.33 MHz. One point for the required period and one for the slack.

32.2-T2. A path has 0.15 ns earliest data arrival and a 0.25 ns hold requirement. What is its hold slack, and does changing period from 8 to 16 ns fix it?

Answer and marking: Slack is -0.10 ns. The same-edge minimum-delay inequality is unchanged by that period increase under the given model. One point for the number and one for the timing explanation.

Practical evidence that is still required

The full transition matrix passes under documented priorities; the worked setup slack is +0.5 ns and hold slack -0.1 ns. The explanation correctly separates minimum and maximum delay and does not claim slower clocking cures hold.

Lesson 32.3 - Binary arithmetic, fixed point and overflow policies

Core quiz - original program questions / 6 points

32.3-C1. What is the signed 8-bit two's-complement range?

Answer: -128 through 127.

32.3-C2. What does stored value 24 mean with scale 1/16?

Answer: 1.5.

32.3-C3. Why use a wider intermediate for multiplication?

Answer: The product can require more bits and has a changed fixed-point scale before rounding or rescaling.

Transfer check - new application questions / 4 points

32.3-T1. Give wrapped and saturated signed eight-bit results for 120+20, then state the range of signed W=16,F=8.

Answer and marking: Wrapped result is -116; saturated result is 127. The fixed-point range is -128 to 127.99609375. One point for the two policies and one for the range with sign included in W.

32.3-T2. With nearest-even division by four, what do products +6, +10, -6 and -10 become?

Answer and marking: +2,+2,-2,-2. These are ties at +/-1.5 and +/-2.5 retained integer units, resolved to the even integer. One point for all results and one for the tie rule.

Practical evidence that is still required

Every four-bit addend pair passes both policies, all directed ties match nearest-even, and clipping counts are reported separately. The exact worked product reconstructs -1.125. A report explains at least one case where reordering saturation changes the answer.

Lesson 32.4 - Clock domains, reset and metastability

Core quiz - original program questions / 6 points

32.4-C1. Does a two-register synchronizer eliminate all metastability risk?

Answer: No. It reduces propagation probability under stated timing and device assumptions.

32.4-C2. Why not independently synchronize every bit of a changing binary bus?

Answer: Bits can be observed from different source transitions, yielding an incoherent word.

32.4-C3. Can ordinary digital simulation prove an analog metastability failure rate?

Answer: No. It does not faithfully model that analog probability and timing behavior.

Transfer check - new application questions / 4 points

32.4-T1. Why does a two-register synchronizer on a one-cycle pulse not guarantee that a slower destination sees the pulse?

Answer and marking: The pulse may rise and fall between destination sampling edges, so neither stage ever samples a high level. Stretch, toggle with rate control, or handshake the event. One point for the missed-sampling mechanism and one for a suitable protocol remedy.

32.4-T2. In a bundled-data handshake, when may the source change the payload, and what additional physical assumption is still required?

Answer and marking: Only after it observes the acknowledgement for the outstanding request, under the declared reset policy. The data must settle at the destination before capture and remain stable for the required window; control synchronization alone does not prove the data-path timing. One point for ownership and one for settling/timing.

Practical evidence that is still required

All chosen phase/ratio cases preserve the specified accepted-word sequence. Reset outcome is deterministic under the declared policy. The report explicitly says the digital protocol model does not prove analog metastability reliability or physical CDC closure.

Lesson 33.1 - RTL semantics and synthesizable SystemVerilog

Core quiz - original program questions / 6 points

33.1-C1. Why assign every combinational output on every path?

Answer: Missing assignments can imply unwanted retained state or latches.

33.1-C2. What does the nonblocking swap example do?

Answer: Each register receives the other register's previous value.

33.1-C3. Does a source-code loop necessarily take multiple hardware clock cycles?

Answer: No. It may elaborate into replicated combinational or parallel hardware.

Transfer check - new application questions / 4 points

33.1-T1. Before an edge a=1 and b=2. What follows a<=b; b<=a, and why?

Answer and marking: After the scheduled nonblocking updates a=2,b=1. Both right-hand sides read the old state. One point for the result and one for the scheduling explanation.

33.1-T2. Why is an omitted else sometimes intentional in always_ff but dangerous in intended combinational logic?

Answer and marking: In a clocked block it commonly means the register holds its state. In combinational logic an unassigned output must retain history, implying a latch instead of a function of current inputs. One point for each context.

Practical evidence that is still required

Swap and counter edge traces match prediction, reset priority is tested and output observations are race-free. No unresolved unintended-latch or multiple-writer issue remains; every skipped tool stage is explicitly marked.

Lesson 33.2 - Testbenches, scoreboards and constrained stimulus

Core quiz - original program questions / 6 points

33.2-C1. Should a scoreboard count attempted or accepted transactions?

Answer: Accepted transactions according to the interface handshake.

33.2-C2. Why save random seeds?

Answer: They allow a failing stimulus sequence to be reproduced.

33.2-C3. Why deliberately inject a known bug?

Answer: It checks that the verification environment can detect the failure it claims to cover.

Transfer check - new application questions / 4 points

33.2-T1. Under this FIFO policy, depth two is full with [A,B], input offers C and output-ready is high. Which transfers occur and what remains?

Answer and marking: Only the read of A is accepted. The queue becomes [B]; C remains pending at the producer because pre-edge input-ready was false. One point for the transfer and one for remaining ownership.

33.2-T2. Why should the scoreboard use an abstract sequence instead of copying the RTL read/write-pointer equations?

Answer and marking: Structural independence reduces the risk that the same pointer or wrap bug appears in the DUT and expected-result code. The scoreboard should model the specified accepted-word order, not reimplement the same microarchitecture. One point for independence and one for the sequence contract.

Practical evidence that is still required

Every directed event is observed at every applicable depth, sequence order is preserved and stalled output remains stable. The legal driver retains a blocked payload. At least one deliberate DUT mutation is detected with a reproducible counterexample.

Lesson 33.3 - Assertions, coverage and formal reasoning

Core quiz - original program questions / 6 points

33.3-C1. Does reaching 100% of chosen coverage bins prove correctness?

Answer: No. The bins may omit important behavior and coverage does not establish every property.

33.3-C2. Why can overconstrained assumptions hide a bug?

Answer: They can exclude the very input sequences that trigger it.

33.3-C3. What limitation must accompany a bounded proof?

Answer: The checked time or state bound, unless an additional argument proves the property unboundedly.

Transfer check - new application questions / 4 points

33.3-T1. Does proving 0<=count<=DEPTH establish FIFO ordering?

Answer and marking: No. Count can stay legal while payloads are swapped, repeated or dropped. Add payload/sequence properties or a tracked-item proof. One point for the limitation and one for an appropriate stronger check.

33.3-T2. A bounded check finds no counterexample through 20 cycles. What claim is justified, and what must not be claimed?

Answer and marking: The specified property passed all allowed modeled traces through the stated 20-cycle bound under its assumptions, subject to the tool result. That is not an unbounded proof or physical timing guarantee. One point for the bounded claim and one for the exclusions.

Practical evidence that is still required

The property ledger separates assumptions, assertions and covers; at least two meaningful covers are reached in an executed run. Any bounded result includes depth, and any unbounded claim includes actual engine output. Ordering is checked, not inferred from count bounds.

Lesson 33.4 - Synthesis, constraints and FPGA resource mapping

Core quiz - original program questions / 6 points

33.4-C1. Does successful synthesis establish timing closure?

Answer: No. Implementation and timing analysis under correct constraints are additional requirements.

33.4-C2. Why inspect inferred memory resources?

Answer: The source description may map to registers or memories with different cost and behavior.

33.4-C3. Why use identical constraints when comparing designs?

Answer: Otherwise reported area or timing differences may reflect the experiment setup rather than the design change.

Transfer check - new application questions / 4 points

33.4-T1. At width 16 and depth eight, how many abstract payload bits are present, and how wide must occupancy be?

Answer and marking: There are 16*8=128 payload bits. Occupancy must represent 0 through 8 and therefore needs four bits; a storage index needs only three. One point for each distinction.

33.4-T2. Why cannot a generic synthesis cell count establish routed FPGA Fmax?

Answer and marking: It lacks the required device-specific mapping, placement, routed interconnect and complete timing constraints/corners. A cell count measures structure, not the maximum constrained path delay. One point for missing physical/constraint context and one for the structure-versus-delay distinction.

Practical evidence that is still required

Two actual synthesis reports have identical documented flow except for the intended variation; functionality is rechecked and memory mapping differences are explained. No generic cell report is called a physical LUT count, power result or routed Fmax.

Lesson 34.1 - Instruction sets, datapaths and single-cycle execution

Core quiz - original program questions / 6 points

34.1-C1. How does an ISA differ from a microarchitecture?

Answer: The ISA specifies programmer-visible behavior; the microarchitecture is one implementation of it.

34.1-C2. What limits a simple single-cycle clock period?

Answer: The longest required combinational path plus sequential timing and margin.

34.1-C3. Why avoid claiming full ISA compliance for a small subset?

Answer: Unsupported instructions or required architectural behaviors may prevent compliance.

Transfer check - new application questions / 4 points

34.1-T1. Why does G16 ADD r3,r1,r2 encode as 0x2650?

Answer and marking: Opcode 2 contributes 0x2000, destination r3 contributes 3<<9=0x600, r1 contributes 1<<6=0x40 and r2 contributes 2<<3=0x10; reserved low bits are zero. Sum is 0x2650. One point for fields and one for the sum.

34.1-T2. What must happen when LD r0 uses an out-of-range address in the G16 contract?

Answer and marking: It faults without retirement or architectural side effects. Discarding a write to r0 does not remove the instruction’s required address validation. One point for the fault behavior and one for the r0 distinction.

Practical evidence that is still required

Arithmetic, load/store and a branch run with matching intermediate state; reserved/illegal encodings and address faults have no unintended side effects. The ISA/microarchitecture distinction and memory assumptions are explicit.

Lesson 34.2 - Pipelining, forwarding, stalls and branch recovery

Core quiz - original program questions / 6 points

34.2-C1. Does pipelining always shorten one instruction's latency?

Answer: No. Its main goal is often improved throughput by overlapping instructions.

34.2-C2. Why must flushed instructions suppress writes?

Answer: They are not part of the accepted instruction path and must not alter architectural state.

34.2-C3. What is a load-use hazard?

Answer: A following instruction needs loaded data before the pipeline has made it available.

Transfer check - new application questions / 4 points

34.2-T1. Why can forwarding the ALU result of a load fail to solve its immediate load-use dependency?

Answer and marking: The ALU result is typically the effective address, not the loaded memory value. The value may not exist until a later stage or response. Stall or forward the actual available load data according to the chosen timing model. One point for address/value and one for timing/remedy.

34.2-T2. A taken branch skips a store, but the final registers match the reference. Is that enough to validate branch recovery?

Answer and marking: No. The wrong-path store may have changed memory despite matching registers. Check memory side effects and valid/write-enable suppression at retirement or acceptance. One point for the missing evidence and one for the required check.

Practical evidence that is still required

Dependent arithmetic, load-use, branch-taken and back-to-back-memory tests match the architectural reference. Invalid or flushed stages cannot write registers or memory. Cycle-count claims name the stage and memory model.

Lesson 34.3 - Memory interfaces, caches and protection boundaries

Core quiz - original program questions / 6 points

34.3-C1. Why is an instantaneous array an incomplete memory model?

Answer: It omits latency, backpressure, errors and device-specific side effects.

34.3-C2. What does a cache tag identify?

Answer: Which memory block is represented by the stored cache line.

34.3-C3. Does a small CPU necessarily need an MMU?

Answer: No, but omitting translation and protection changes its capabilities and safe use cases.

Transfer check - new application questions / 4 points

34.3-T1. A 32-bit byte address uses 16-byte blocks and 64 direct-mapped lines. Give offset, index and tag widths.

Answer and marking: Offset is log2(16)=4 bits, index log2(64)=6 bits, and tag 32-4-6=22 bits. One point for offset/index and one for tag plus byte-address assumption.

34.3-T2. A store request is accepted once, then its response is delayed five cycles. How many times may the memory write occur?

Answer and marking: Exactly once under this store contract. Waiting cycles are not additional accepted transactions; the sender must not re-offer the same request as new work after acceptance. One point for the count and one for the protocol reasoning.

Practical evidence that is still required

Programs preserve architectural results under randomized stalls; every accepted store has exactly the specified number of side effects. Request payload remains stable while blocked, errors do not retire successfully and reset ownership is documented. Any included cache passes conflict tests.

Lesson 34.4 - Processor verification, toolchains and architectural evidence

Core quiz - original program questions / 6 points

34.4-C1. Why compare at retirement rather than arbitrary internal cycles?

Answer: Different correct implementations can have different internal timing while preserving the same architectural effects.

34.4-C2. Why inspect compiler-generated instructions?

Answer: The compiler or runtime may use operations unsupported by the educational core.

34.4-C3. What belongs in a reproducible processor failure report?

Answer: Program bytes, initial state, toolchain/configuration, seed and the first architectural mismatch.

Transfer check - new application questions / 4 points

34.4-T1. Why compare retirement events rather than register files after the same number of cycles in a reference interpreter and pipeline?

Answer and marking: They can implement the same instruction semantics at different cycle timings and with different bubbles/stalls. Retirement aligns committed architectural events. One point for timing differences and one for the correct comparison boundary.

34.4-T2. Does generating random legal programs with the same decoder code used by the DUT provide fully independent verification?

Answer and marking: No. Shared interpretation or field-extraction bugs can affect both stimulus and DUT. Add independently specified generation, hand-encoded cases and an independent semantic reference. One point for correlated-error risk and one for a remedy.

Practical evidence that is still required

At least twenty directed programs and a reproducible randomized set match between DUT and independent reference; mismatch localization works and an assembled program runs end to end. Timeouts and unsupported cases are not counted as passes.

Lesson 35.1 - Memory-mapped I/O, timers, interrupts and DMA

Core quiz - original program questions / 6 points

35.1-C1. Why are device registers not ordinary memory variables?

Answer: Accesses can have side effects, ordering requirements and restricted widths.

35.1-C2. When may software reuse a DMA source buffer?

Answer: After the device has completed the access and ownership is returned under the protocol.

35.1-C3. Does C++ volatile make a shared device driver thread-safe?

Answer: No. It is not a substitute for synchronization or the platform's device-access rules.

Transfer check - new application questions / 4 points

35.1-T1. With pending=1, clear=1 and event=1, what does set-wins W1C logic produce, and why?

Answer and marking: The next pending bit is (1 & ~1)|1 = 1, preserving the newly arriving event. One point for the result and one for the event-loss rationale.

35.1-T2. Can holding a device response valid for five cycles justify repeating the original enqueue five times?

Answer and marking: No. The request acceptance caused one side effect. Response backpressure only delays response consumption; it must not create new accepted requests or duplicate effects. One point for exactly-once semantics and one for separating the two channels.

Practical evidence that is still required

The timer event occurs on the defined edge, clear-plus-event preserves the new pending status, and every MMIO side effect occurs once. The ownership model rejects early reuse and the report does not confuse volatile with DMA coherence.

Lesson 35.2 - Framebuffers, scanout and pixel pipelines

Core quiz - original program questions / 6 points

35.2-C1. How many bytes does the example framebuffer require?

Answer: 320 x 240 x 2 = 153,600 bytes.

35.2-C2. Is active-pixel bandwidth the whole physical interface bandwidth?

Answer: No. Blanking, encoding, protocol and other traffic can add overhead.

35.2-C3. Why synchronize framebuffer swaps?

Answer: An arbitrary swap can display parts of different frames, causing tearing.

Transfer check - new application questions / 4 points

35.2-T1. For 64x48 active pixels inside 80x60 total periods at 288 kHz, find frame frequency and active samples per frame.

Answer and marking: Frame frequency is 288000/(80*60)=60 Hz. Active samples are 64*48=3072. One point for each result and the distinction between total and active.

35.2-T2. Why can delaying RGB by one register while leaving active unchanged corrupt an otherwise correct frame?

Answer and marking: Color now belongs to the preceding coordinate while active labels the current one, shifting or dropping boundary pixels. Delay the associated sideband signals or define and implement an equivalent alignment. One point for the mismatch and one for the remedy.

Practical evidence that is still required

Each educational frame has 4,800 periods and 3,072 active pixels with exact wrap. Captured active pixels match the reference and any buffer swap occurs only at the declared boundary. Physical video compatibility is not claimed from these timings.

Lesson 35.3 - A small accelerator and its software contract

Core quiz - original program questions / 6 points

35.3-C1. What total time follows from halving the example 20 microsecond compute phase?

Answer: 80 + 10 = 90 microseconds.

35.3-C2. What does backpressure accomplish?

Answer: It prevents a producer from overrunning a consumer that cannot currently accept more data.

35.3-C3. Why report latency and throughput separately?

Answer: A pipeline can accept frequent work while each individual item still takes multiple cycles to complete.

Transfer check - new application questions / 4 points

35.3-T1. Software costs 2 us+8N ns and hardware 30 us+3N ns. What is the smallest integer N for a strict modeled hardware win?

Answer and marking: N>5600, so 5601. At 5600 the paths tie. One point for the inequality and one for the strict integer boundary.

35.3-T2. If 20% of application time speeds up 4x, what is the ideal total speedup before new overhead?

Answer and marking: 1/(0.8+0.2/4)=1/0.85=1.17647. One point for weighting the unchanged portion and one for the result/overhead limitation.

Practical evidence that is still required

Outputs match the bit-accurate reference; order, no-duplication and stalled-data stability hold. The worked cost model ties at N=5,600, and modeled versus physical timing is explicit. Any speedup claim includes overhead and application fraction.

Lesson 35.4 - Integrated digital-design review and optional board bring-up

Core quiz - original program questions / 6 points

35.4-C1. Why are unit tests insufficient for integration?

Answer: Connected components may disagree on protocols, reset, timing or ownership despite working in isolation.

35.4-C2. What must be checked before wiring a peripheral?

Answer: Pin assignment, voltage compatibility, interface requirements and the reviewed board safety plan.

35.4-C3. Does a successful FPGA demonstration prove an ASIC is ready to fabricate?

Answer: No. ASIC physical design, verification, manufacturing and signoff impose additional requirements.

Transfer check - new application questions / 4 points

35.4-T1. Can a sprite-compositor capstone replace the required Module 34 processor exercise?

Answer and marking: No. Gate G can center on a different subsystem, but the module-level processor model/RTL and verification work remain required. One point for the answer and one for distinguishing module work from the capstone choice.

35.4-T2. A generic synthesis run succeeds and a board later displays an image. What stronger claims still need separate evidence?

Answer and marking: Target timing closure, power/reliability under defined conditions, full interface correctness and ASIC readiness are not established merely by those outcomes. Name the needed constraints, verification and physical signoff for the specific claim. One point for at least two exclusions and one for explaining the missing evidence.

Practical evidence that is still required

Gate G evidence and rubric pass with no critical correctness, reproducibility or claim-integrity failure. The module-level CPU requirement is separately complete. Board operation is optional; actual core simulation and synthesis are not.